

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO1	Assessment:	Assessment Tool 2 - Concept Mapping
Weightage:	35%	Total Marks:	25

CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
-----------------------	--

Student Name:	G Kushwanth	Reg. No.:	192424150
Department:		Date:	

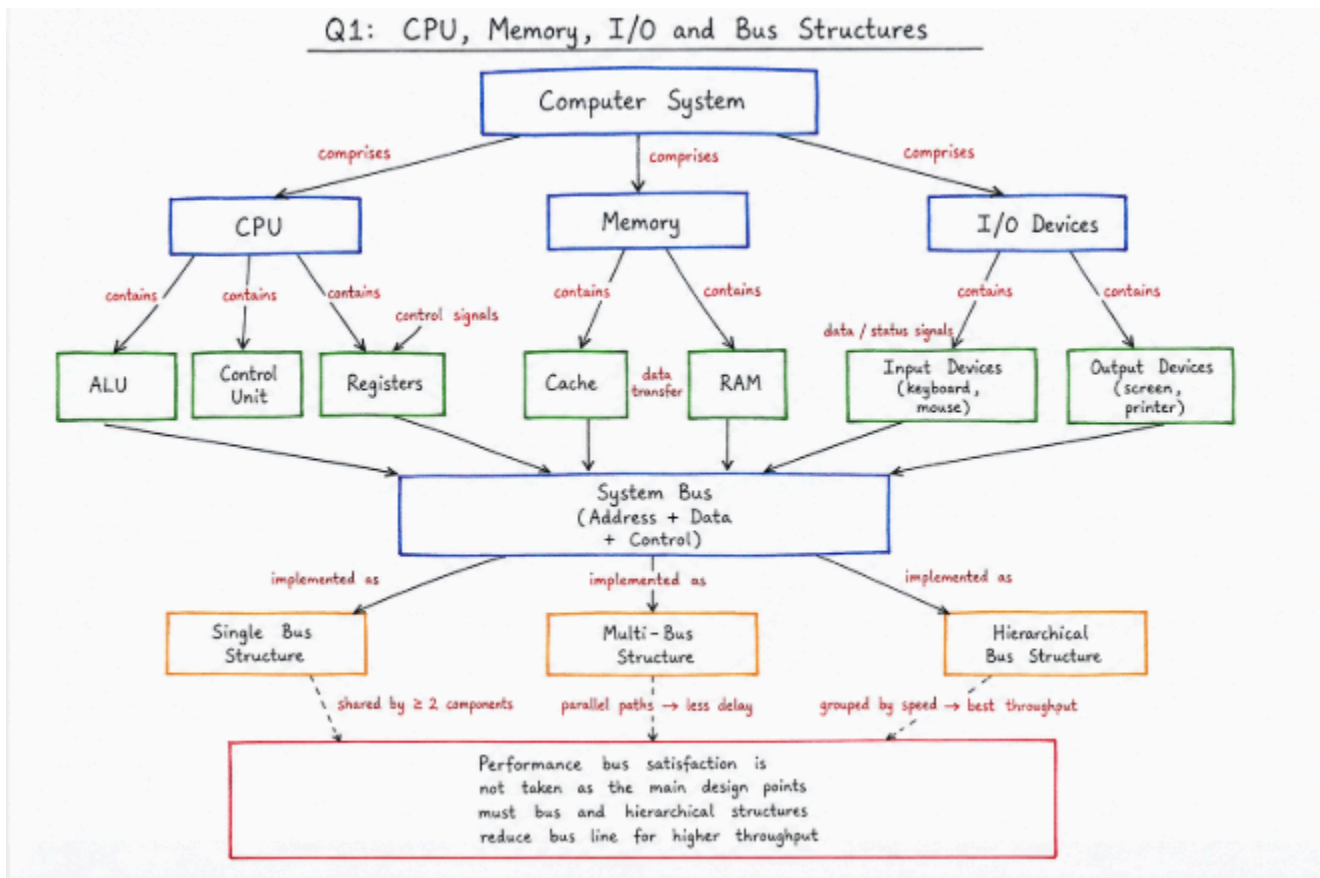
Annexure A — Concept Map Tasks & Responses

The five concept-mapping tasks below correspond to CO1 (Analyze the functional units of a computer system). Each response includes the original task prompt, a visual concept map, and a short written explanation of the relationships depicted.

Question 1

Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.

Concept Map



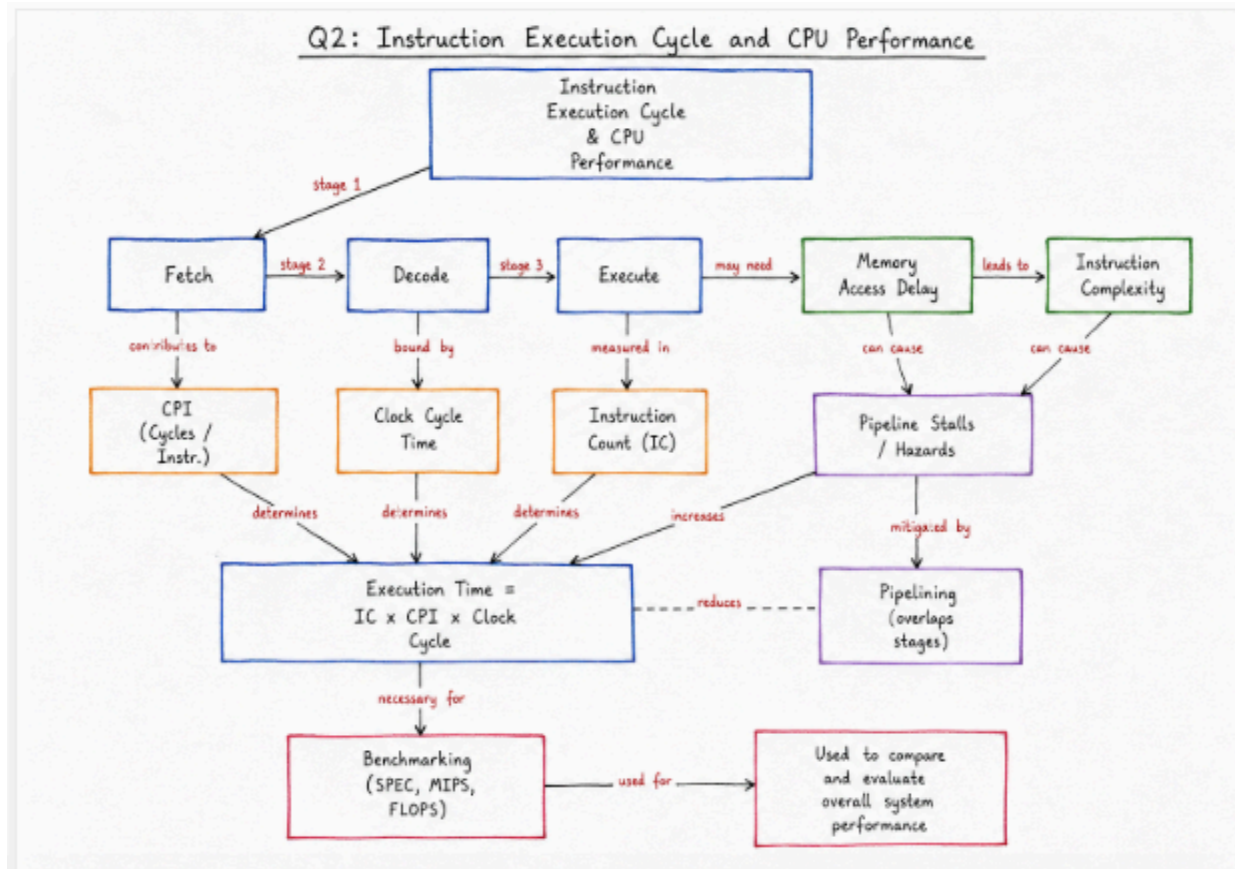
Explanation

The computer system is organized into three functional units — the CPU (ALU, Control Unit, Registers), Memory (Cache, RAM) and I/O devices — all of which communicate through the System Bus, which carries address, data and control signals. The CPU issues control signals that coordinate data transfer between memory and I/O. A single-bus structure is the simplest but forces all units to share one communication path, creating contention and bottlenecks when multiple devices need to transfer data simultaneously. A multi-bus structure adds parallel paths (e.g., separate address/data buses) to reduce this contention, while a hierarchical bus structure groups devices by speed (fast CPU-cache bus, slower peripheral bus) so that high-speed components are not held back by slow I/O devices. This progression from single to hierarchical buses directly improves overall system throughput and reduces waiting time caused by bus arbitration.

Question 2

Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.

Concept Map



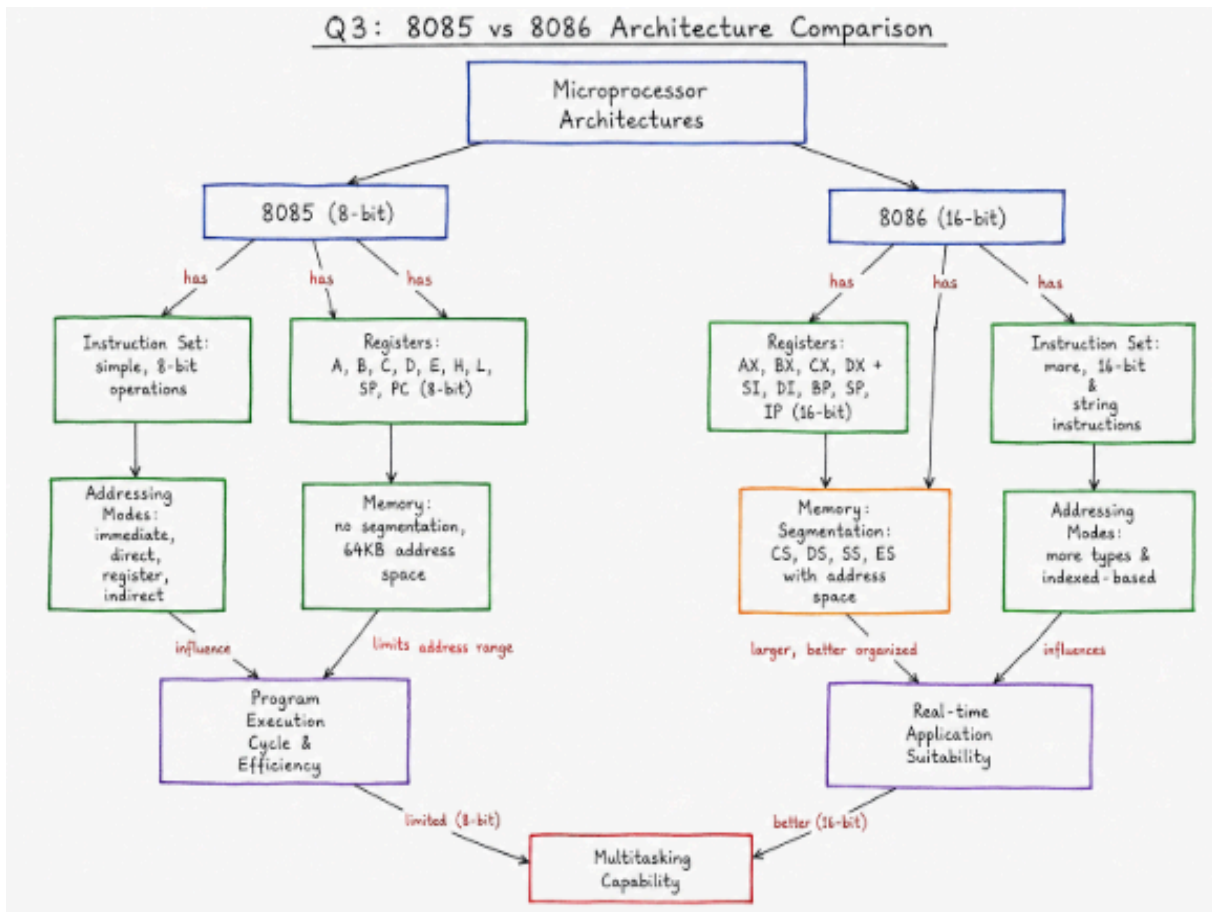
Explanation

Every instruction passes through the Fetch, Decode and Execute stages. Each stage consumes clock cycles, and the number of cycles an instruction needs is captured by CPI (Cycles Per Instruction). The fundamental performance equation, $\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$, ties these stages directly to overall system speed. Memory access delays and instruction complexity increase the effective CPI by causing pipeline stalls and hazards, which pipelining techniques attempt to hide by overlapping stages of successive instructions. Benchmarking suites (such as SPEC or MIPS/FLOPS ratings) are then used to measure the resulting execution time across representative workloads, allowing designers and users to compare the real-world performance of different CPUs.

Question 3

Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

Concept Map



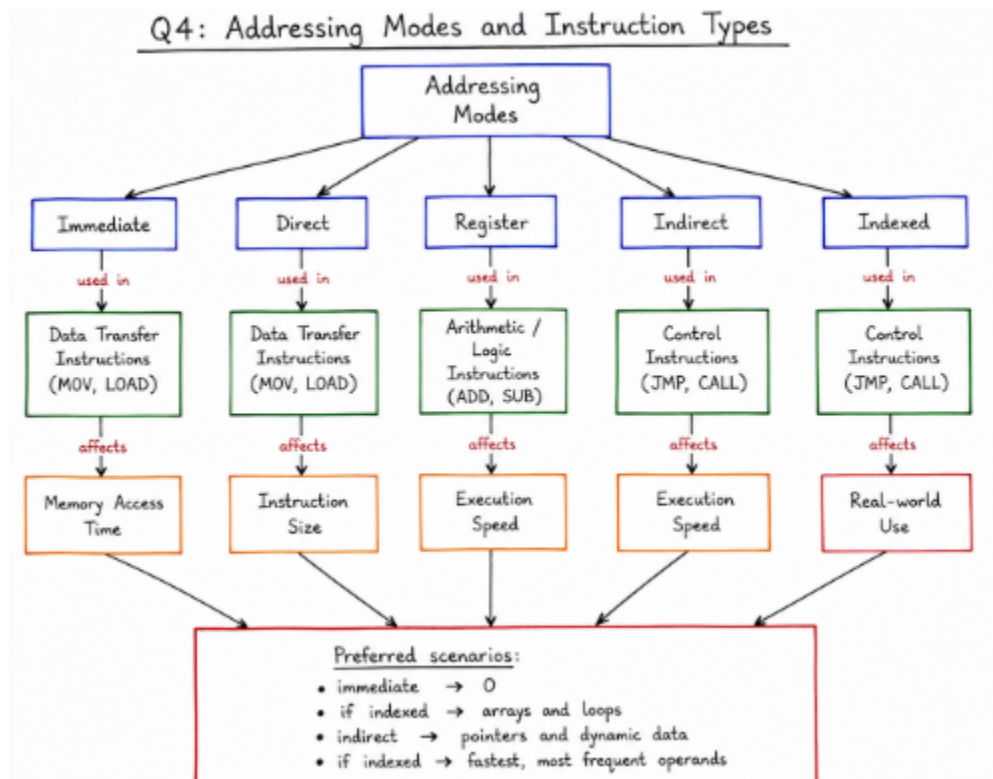
Explanation

The 8085 is an 8-bit microprocessor with a simpler instruction set, an 8-bit register set (A, B, C, D, E, H, L), basic addressing modes and no memory segmentation, limiting it to a 64KB direct address space. The 8086, in contrast, is a 16-bit processor with a richer instruction set (including string instructions), 16-bit general-purpose and dedicated segment registers (CS, DS, SS, ES), additional indexed/based-indexed addressing modes, and memory segmentation that extends its addressable space to 1MB. This segmentation and the wider register/addressing support let the 8086 execute larger and more complex programs efficiently and provide the foundation needed for multitasking, whereas the 8085's flat, small address space restricts it mainly to simple, single-task, real-time control applications.

Question 4

Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.

Concept Map



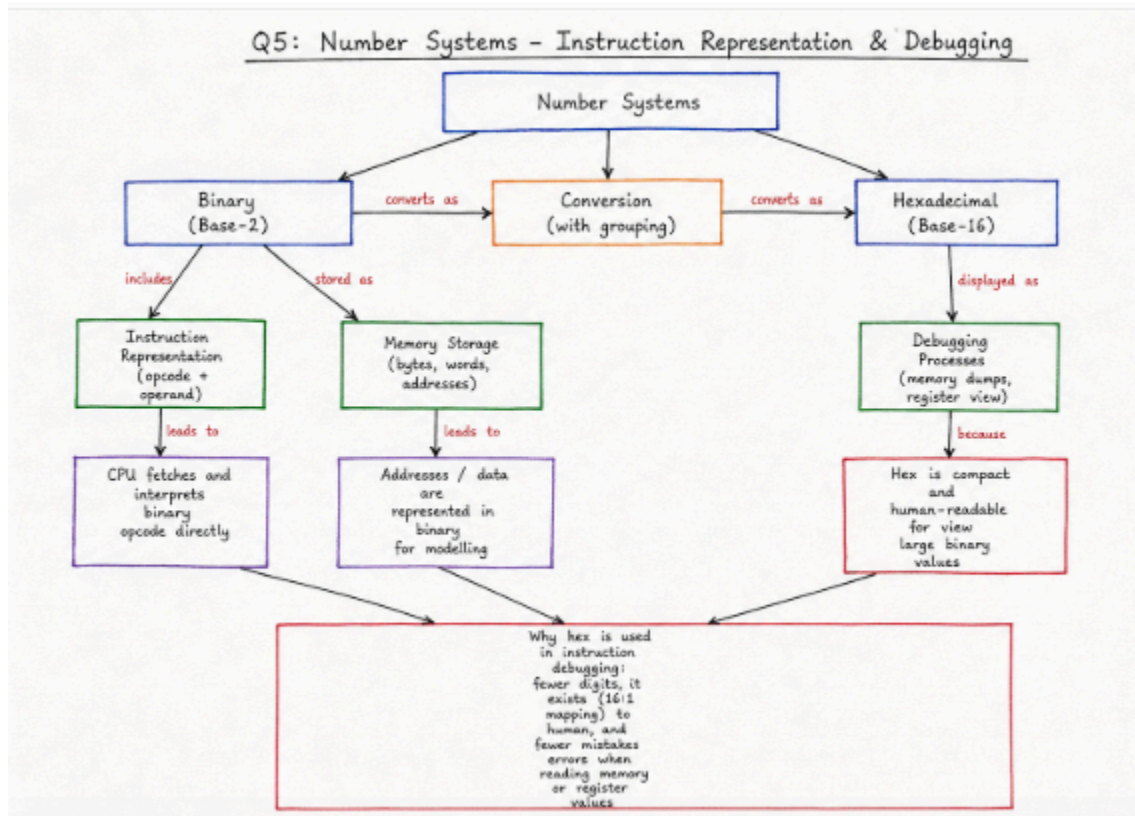
Explanation

Addressing modes determine how an instruction's operand is located. Immediate and Direct modes are commonly used in data transfer instructions since the operand or its address is readily available, giving the fastest execution with minimal memory references. Register and Indirect modes are frequently used in arithmetic/logic instructions, trading a small amount of extra access time for flexibility in operand location. Indexed and Indirect modes are essential for control instructions and for accessing arrays or data structures via pointers, at the cost of larger instruction size and additional memory access time. In practice, immediate addressing is preferred for constants, indexed addressing for looping through arrays, and indirect addressing for pointer-based or dynamic data access — so the choice of addressing mode is a direct trade-off between instruction size, memory access time and execution speed.

Question 5

Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

Concept Map



Explanation

Binary (base-2) is the native language of the CPU: every instruction opcode and operand is ultimately stored and interpreted as binary in memory. Hexadecimal (base-16) is directly related to binary through simple 4-bit grouping, making conversion between the two straightforward. While the CPU fetches and executes binary opcodes directly, humans reading memory dumps, register contents or debugging output find long binary strings error-prone and hard to interpret. Hexadecimal compresses these long strings into far fewer digits while preserving an exact, easy mapping to the underlying binary, which is why it is the standard representation used in debuggers, memory maps and low-level system design documentation — it reduces human error without changing what the CPU actually processes.

Code of Conduct

I G.Kushwanth(192424150) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: g kuswanth